

2. 目标程序的优化

目标程序是由指令直接组成的，可以在处理机中直接执行，而面向目标程序优化指令系统是提高计算机系统性能的最直接的方法。优化目标程序主要有两个目的：一是缩短程序的长度，即减少程序的空间开销；另一个是缩短程序的执行时间，即减少程序的时间开销。

优化目标程序的主要途径是增强指令的功能，包括数据传送类指令、运算类指令和程序控制类指令。具体方法为：对大量的程序及其执行情况进行统计分析，找出使用频率高、执行时间长的指令和指令串；对于使用频率高的指令，用硬件加快其执行，以缩短整个程序的执行时间；对于使用频率高的指令串，用一条新的指令来代替它，这样既能缩短整个程序的执行时间，也能缩短整个程序的长度，从而减少程序的空间开销。

3. 对高级语言和编译程序的支持

用高级语言编写的程序经编译程序编译后生成的目标程序，与直接用机器语言或汇编语言编写的程序相比，时空开销都要大得多。改进指令系统，增加对高级语言和编译程序的支持，缩小高级语言与机器语言之间的差距，从而提高整个计算机系统的性能。

面向高级语言和编译程序增强指令系统的途径主要有两个：一是增强对高级语言和编译程序支持的指令的功能，增强体系结构的规整性，减少体系结构中各种例外情况；二是设计高级语言计算机，在这种计算机中，用高级语言编写的程序不需要经过编译，直接由机器的硬件来执行，例如，LISP 计算机和 PROLOG 计算机等。

4. CISC 的缺陷

CISC 主要存在如下三个方面的问题：

(1) 80-20 规律。在 CISC 中，指令使用频率不均衡，即各种指令的使用频率相差悬殊。大量统计数字表明，大约有 20% 的指令使用频率比较高，占据了 80% 的处理机时间。换句话说，有 80% 的指令只在 20% 的处理机运行时间内才被用到。

(2) CISC 结构指令系统的复杂性给 VLSI（超大规模）设计增加了负担，不利于单片集成。但随着 VLSI 集成度迅速提高，生产单芯片处理机已成为现实。在单芯片处理机内，希望采用规整的硬布线控制逻辑，而不是用微程序。

(3) 软硬件的功能分配问题。为了支持目标程序的优化，支持高级语言和编译程序，在 CISC 中增加了许多复杂的指令。这些复杂指令简化了目标程序，缩小了高级语言与机器指令之间的语义差距，但这是以增加硬件的复杂程度、指令执行周期长为代价的。

3.4.3 精简指令系统

精简指令系统计算机（Reduced Instruction Set Computer, RISC）简称精简指令系统，该系统不是简单地把指令系统进行简化，而是通过简化指令的途径使计算机的结构更加简单、合理，以减少指令的执行周期数，从而提高运算速度。

1. RISC 的特点

RISC 要求指令系统简化，操作在单周期内完成，指令格式力求一致，寻址方式尽可能减少，

并提高编译的效率，最终达到加快机器处理速度的目的。RISC 的主要特点如下：

(1) 指令数量少。优先选取使用频率最高的一些简单指令和常用指令，避免使用复杂指令。只提供了 LOAD（从存储器中读数）和 STORE（把数据写入存储器）两条指令对存储器操作，其余所有的操作都在 CPU 的寄存器之间进行。

(2) 指令的寻址方式少。通常只支持寄存器寻址、立即数寻址和相对寻址方式。

(3) 指令长度固定，指令格式种类少。RISC 指令数量少、格式种类少、相对简单，其指令长度固定，指令之间各字段的划分比较一致，译码相对容易。

(4) 以硬布线逻辑控制为主。为了提高操作的执行速度，通常采用硬布线逻辑（组合逻辑）来构建控制器。

(5) 单周期指令执行，采用流水线技术。因为简化了指令系统，很容易利用流水线技术，使得大部分指令都能在一个机器周期内完成。少数指令可能需要多周期，例如，LOAD/STORE 指令因为需要访问存储器，其执行时间就会长一些。

(6) 优化的编译器。RISC 的精简指令集使编译工作简单化。因为指令长度固定、格式种类少、寻址方式少，编译时不必在具有相似功能的许多指令中进行选择，也不必为寻址方式的选择而费心，同时易于实现优化，从而可以生成高效率执行的机器代码。

(7) CPU 中的通用寄存器数量多，一般在 32 个以上，有的可达上千个。

大多数 RISC 采用了 Cache 方案，使用 Cache 来提高取指令的速度。而且，有的 RISC 使用两个独立的 Cache 来改善性能，一个称为指令 Cache，另一个称为数据 Cache。这样，取指令和取数据可以同时进行，互不干扰。

2. RISC 的核心思想

计算机执行一个程序所用的时间可表示为：

$$P = I \times \text{CPI} \times T$$

其中， P 是执行这个程序所使用的总时间， I 是这个程序所需执行的总的指令条数，CPI 是每条指令执行的平均周期数， T 是一个周期的时间长度。

(1) 由于 RISC 的指令都比较简单，CISC 中的一条复杂指令所完成的功能在 RISC 中可能要用几条指令才能实现。对于同一个源程序，编译后生成的动态目标代码，显然 RISC 的要比 CISC 的多。但是，由于 CISC 中复杂指令使用的频率很低，程序中使用的绝大多数指令都是与 RISC 一样的简单指令，因此，实际上的统计结果表明，RISC 的 I 长度只比 CISC 的长 20% ~ 40%。

(2) 由于 CISC 一般是用微程序实现的，一条指令往往要用好几个周期才能完成，一些复杂指令所需要的周期数就更多。根据统计，大多数 CISC 的指令平均执行周期数（CPI）为 4 ~ 10；而 RISC 的大多数指令都是单周期执行的，它们的 CPI 应该是 1，但是，由于 RISC 中还有 LOAD 和 STORE 指令，以及少数复杂指令，所以，其 CPI 要略大于 1。

(3) 由于 RISC 一般采用硬布线逻辑实现，指令要实现的功能都比较简单，所以，CISC 的 T 通常是 RISC 的 3 倍左右。

综合以上三点，可以大致计算出，RISC 的处理速度要比同规模的 CISC 提高 3 ~ 5 倍。其中的关键在于 RISC 的指令平均执行周期数（CPI）减小了，这正是 RISC 设计思想的精华。减

小 CPI 是多个方面共同努力的结果。在硬件方面,采用硬布线逻辑控制,减少指令和寻址方式的种类,使用固定的指令格式,采用 LOAD/STORE 结构,指令执行过程中设置多级流水线等;软件方面则十分强调优化编译技术的作用。

3. RISC 的关键技术

RISC 要达到很高的性能,必须有相应的技术支持。目前,在 RISC 处理机中采用的主要技术有如下几种:

(1) 延迟转移技术。在 RISC 处理机中,指令一般采用流水线方式工作。流水线技术所面临的一个问题就是转移指令的出现,这样有可能使流水线断流。其解决办法是采用延迟转移技术,该方法是在转移指令之后插入一条有效的指令,使得转移指令好像被延迟执行了。

(2) 指令取消技术。采用指令延迟技术,当遇到条件转移指令时,调整指令序列非常困难,在许多情况下找不到可以用来调整的指令。有些 RISC 处理机采用指令取消技术。在使用指令取消技术的处理机中,所有转移指令和数据变换指令都可以决定下面待执行的指令是否应该取消。如果指令被取消,其效果相当于执行了一条空操作指令,不影响程序的运行环境。为了提高程序的执行效率,应该尽量少取消指令,以保持指令流水线处于充满状态。

(3) 重叠寄存器窗口技术。在处理机中设置一个数量比较大的寄存器堆,并把它划分成很多个窗口。每个过程使用其中相邻的三个窗口和一个公共的窗口,而在这些窗口中,有一个窗口是与前一个过程共用的,还有一个窗口是与下一个过程共用的。与前一个过程共用的窗口可以用来存放前一个过程传送给本过程的参数,同时也存放本过程传送给前一个过程的计算结果;与下一个过程共用的窗口可以用来存放本过程传送给下一个过程的参数,同时也存放下一个过程传送给本过程的计算结果。

(4) 指令流调整技术。为了使 RISC 处理机中的指令流水线高效率地工作,尽量不断流,编译器必须分析程序的数据流和控制流,当发现指令流有断流可能时,要调整指令序列。对有些可以通过变量重新命名来消除的数据相关,要尽量消除。这样,可以提高流水线的执行效率,缩短程序的执行时间。

(5) 逻辑实现以硬件为主,固件为辅。RISC 主要采用硬布线逻辑来实现指令系统。对于那些必需的复杂指令,也可用固件实现。

在实际应用中,商品化的 RISC 机器并不是纯粹的 RISC。为了满足应用的需要,实际应用的 RISC 除了保持 RISC 的基本特色之外,还必须辅以一些必不可少的复杂指令(例如,浮点运算、十进制运算指令等)。这种机器实际上是在 RISC 基础上实现了 RISC 与 CISC 的完美结合。

3.5 多处理机系统

流水线计算机通过多级流水的同时操作来获得高性能,但只能执行单个程序。而且,由于器件本身的限制,任何处理机的速度都是有限的。要想突破这个限制,就必须使用多个处理机并行执行。多处理机结构由若干个独立的处理机组成,每个处理机能够独立执行自己的程序。多处理机结构属于 MIMD 结构,处理机之间按某种形式互连,从而实现程序之间的数据交换和同步。